

OSSP-PRACTICAL-WEEK8

TASK1: Develop a program using `malloc()`, `calloc()`, `realloc()`, and `free()`. Monitor memory allocation behavior and identify memory leaks using Valgrind.

Source code:

```
GNU nano 2.9.2
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    printf("Dynamic Memory Allocation Demo\n");
    printf("-----\n");

    // malloc()
    int *arr = (int *)malloc(3 * sizeof(int));

    if (arr == NULL)
    {
        printf("malloc failed.\n");
        return 1;
    }

    for (i = 0; i < 3; i++)
        arr[i] = (i + 1) * 10;

    printf("\nAfter malloc():\n");
    for (i = 0; i < 3; i++)
        printf("%d ", arr[i]);

    // calloc()
    int *zero_arr = (int *)calloc(3, sizeof(int));

    if (zero_arr == NULL)
    {
        printf("\ncalloc failed.\n");
        free(arr);
        return 1;
    }

    printf("\n\nAfter calloc():\n");
    for (i = 0; i < 3; i++)
        printf("%d ", zero_arr[i]);

    // realloc()
```

```

// realloc()
arr = (int *)realloc(arr, 5 * sizeof(int));

if (arr == NULL)
{
    printf("\nrealloc failed.\n");
    free(zero_arr);
    return 1;
}

arr[3] = 40;
arr[4] = 50;

printf("\n\nAfter realloc():\n");
for (i = 0; i < 5; i++)
    printf("%d ", arr[i]);

// free()
free(arr);
free(zero_arr);

printf("\n\nMemory released using free().\n");

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano practical_week8_task1.c
root@LAPTOP-1COJKKG6:~# gcc practical_week8_task1.c -o practical_week8_task1
root@LAPTOP-1COJKKG6:~# ./practical_week8_task1
Dynamic Memory Allocation Demo
-----

After malloc():
10 20 30

After calloc():
0 0 0

After realloc():
10 20 30 40 50

Memory released using free().
root@LAPTOP-1COJKKG6:~# |

```

```

Dynamic Memory Allocation Demo
-----

After malloc():
10 20 30

After calloc():
0 0 0

After realloc():
10 20 30 40 50

Memory released using free().
==1905==
==1905== HEAP SUMMARY:
==1905==      in use at exit: 0 bytes in 0 blocks
==1905==    total heap usage: 4 allocs, 4 frees, 1,068 bytes allocated
==1905==
==1905== All heap blocks were freed -- no leaks are possible
==1905==
==1905== For lists of detected and suppressed errors, rerun with: -s
==1905== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how dynamic memory allocation works using malloc(), calloc(), realloc(), and free(). I understood how memory can be allocated, resized, initialized, and released dynamically. I also learned how Valgrind can be used to detect memory leaks and check whether allocated memory is properly released.

Task2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main()
{
    int *data;
    pid_t pid;

    data = malloc(SIZE * sizeof(int));

    if (data == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < SIZE; i++)
        data[i] = 10;

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated for %d integers.\n", SIZE);

    pid = fork();

    if (pid < 0)
    {
        printf("fork failed.\n");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild process started.\n");
        printf("Child PID: %d\n", getpid());

        printf("Before modification: data[0] = %d\n", data[0]);

        // Modify memory in child
        data[0] = 999;

        printf("After modification: data[0] = %d\n", data[0]);

```

```

        // Modify memory in child
        data[0] = 999;

        printf("After modification: data[0] = %d\n", data[0]);

        printf("Child modified the data.\n");

        sleep(5);

        free(data);
        exit(0);
    }
    else
    {
        printf("\nParent process.\n");
        printf("Child PID: %d\n", pid);

        sleep(2);

        printf("Parent data[0] = %d\n", data[0]);

        wait(NULL);

        printf("Child process completed.\n");

        free(data);
    }

    return 0;
}

```

Output:

```
root@LAPTOP-1C0JKKG6:~# nano practical_week8_task2.c
root@LAPTOP-1C0JKKG6:~# gcc practical_week8_task2.c -o practical_week8_task2
root@LAPTOP-1C0JKKG6:~# ./practical_week8_task2
Parent PID: 1924
Memory allocated for 1000000 integers.

Parent process.
Child PID: 1925

Child process started.
Child PID: 1925
Before modification: data[0] = 10
After modification: data[0] = 999
Child modified the data.
Parent data[0] = 10
```

Observation: I learned how Copy-on-Write works after a fork(). I understood that the parent and child initially share memory pages, and when the child modifies the data, Linux creates a separate copy for the child. I also learned how to observe process memory usage using /proc and process monitoring commands.